

Sistemet e Bazës së të Dhënave

Modeli Fizik (DDL - Gjuha për Definimin e të Dhënave)

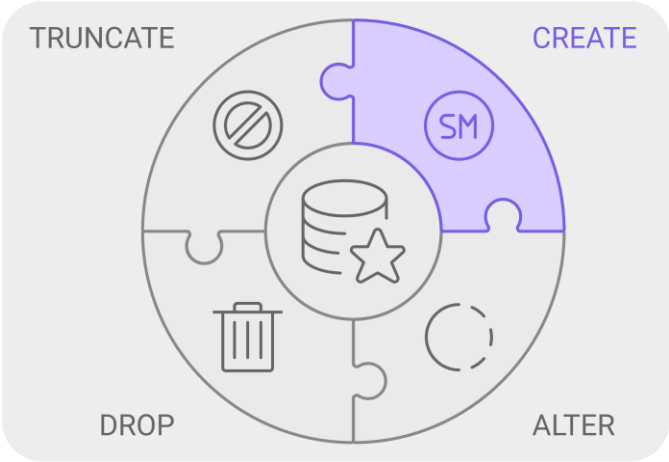
Elton Boshnjaku

Çfarë është DDL?

DDL (Data Definition Language) është **grup komandash në SQL** që përdoren për të **krijuar**, **ndryshuar** dhe **fshirë strukturat bazë të të dhënave**, si tabelat dhe skemat.

Komandat kryesore përfshijnë:

- CREATE (për të krijuar objekte të reja),
- ALTER (për të modifikuar ekzistueset),
- DROP (për të fshirë objekte), dhe
- TRUNCATE (për të fshirë të dhënat).



Rëndësia e DDL

Ndihmon në **përcaktimin e strukturës së bazës së të dhënave**, **duke garantuar organizimin** dhe **qartësinë** e të dhënave.

Vendos **rregulla të integritetit**, **duke siguruar që të dhënat të jenë të sakta** dhe në përputhje me kërkesat.

Fokusi Kryesor i DDL

Krijimi i tabelave dhe vendosja e kufizimeve:

DDL lejon shtimin e rregullave për të kontrolluar vlerat që futen në bazën e të dhënave.

Integriteti referencial:

Siguron që lidhjet midis tabelave, si çelësat primarë dhe të huaj, të jenë të sakta dhe të qëndrueshme.

Kardinaliteti dhe marrëdhëniet mes entiteteve:

Përkufizon raportet në mes të tabelave, duke ndihmuar në modelimin e saktë të të dhënave dhe ruajtjen e tyre.

Komanda ‘Create’

Qëllimi: Përdoret për të krijuar tabela ose objekte të tjera në bazën e të dhënave.

Shembull:

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL,
    Salary DECIMAL(10, 2) CHECK (Salary > 0)
);
```

Komanda ‘Alter’

Qëllimi: Modifikon strukturën e një tabele ekzistuese, duke shtuar ose ndryshuar kolona.

Shembull:

```
ALTER TABLE Employees
ADD DateOfBirth DATE;
```

Komanda ‘Drop’

Qëllimi: Fshin një tabelë ose objekt tjetër nga baza e të dhënave, duke fshirë të gjitha të dhënat dhe strukturën.

Shembull:

```
DROP TABLE Employees;
```

Komanda ‘Truncate’

Qëllimi: Fshin të gjitha rreshtat nga një tabelë, duke ruajtur strukturën e saj.

Shembull:

```
TRUNCATE TABLE Employees;
```



Data Types

Numeric Data Types

Tipet e të Dhënave Numerike:

Data Type	From	To
BigInt	-2^{63} (-9,223,372,036,854,775,808)	2^{63} -1 (9,223,372,036,854,775,807)
Int	-2^{31} (-2,147,483,648)	2^{31}-1 (2,147,483,647)
smallint	-2^{15} (-32,768)	2^{15}-1 (32,767)
tinyint	0	2^{8}-1 (255)
bit	0	1
decimal	-10^{38}+1	10^{38}-1
numeric	-10^{38}+1	10^{38}-1
money	-922,337,203,685,477.5808	922,337,203,685,477.5807
smallmoney	-214,748.3648	214,748.3647

decimal(p,s) - **p (precision)** që përcakton numrin total të shifrave dhe **s (scale)** që tregon numrin e shifrave pas presjes dhjetore

Character String Data Types

Tipet e të Dhënave për karaktere:

Data Type	Description
char	The maximum length of 8000 characters.(Fixed-Length non-Unicode Characters)
varchar	The maximum length of 8000 characters.(Variable-Length non-Unicode Characters)
varchar(max)	The maximum length of 231 characters(SQL Server 2005 only).(Variable Length non-Unicode data)
text	The maximum length of 2,127,483,647 characters(Variable Length non-Unicode data)

Unicode Character String Data Types

Tipet e të dhënave për vargjet **Unicode**, si nchar, nvarchar, dhe nvarchar(max), **mbështesin karakteret Unicode dhe përdorin 2 byte për karakter**, duke ofruar mbështetje për gjuhë të ndryshme; ndryshe nga char dhe varchar, të cilat përdorin vetëm 1 byte për karakter dhe nuk mbështesin plotësisht karakteret Unicode.

Tipet e të Dhënave për karaktere Unicode:

Data Type	Description
nchar	The maximum length of 4000 characters(Fixed-Length Unicode Characters)
Nvarchar	The maximum length of 4000 characters.(Variable-Length Unicode Characters)
nvarchar(max)	The maximum length of 231 characters(SQL Server 2005 only).(Variable Length Unicode data)

Date and Time Data Types

Tipet e të Dhënave për datë dhe kohë janë:

Data Type	Description
DATE	A data type is used to store the data of date in a record
TIME	A data type is used to store the data of time in a record
DATETIME	A data type is used to store both the data,date, and time in the record.

MSSQL Data Types and Their Sizes

Data Type	Size	Data Type	Size
bigint	8 bytes	**int**	4 bytes
smallint	2 bytes	**tinyint**	1 byte
bit	1 bit	**decimal(p,s)**	5–17 bytes (varies)
numeric(p,s)	Same as decimal	**money**	8 bytes

Data Type	Size	Data Type	Size
smallmoney	4 bytes	**char(n)**	`n` bytes (fixed)
varchar(n)	`n + 2` bytes	**varchar(max)**	Up to 2 GB
text	Up to 2 GB	**nchar(n)**	`n × 2` bytes
nvarchar(n)	`n × 2 + 2` bytes	**nvarchar(max)**	Up to 2 GB
date	3 bytes	**time**	3–5 bytes (precision)
datetime	8 bytes		

Kufizimet - Constraints

Çfarë janë Kufizimet?

Kufizimet (Constraints) janë **rregulla** të vendosura mbi kolona ose tabela **për të siguruar integritetin dhe cilësinë e të dhënave** në një bazë të dhënash.

Ato **ndihmojnë në kontrollimin e vlefshmërisë, konsistencës dhe strukturës së të dhënave**.

Llojet Kryesore të Kufizimeve:

NOT NULL

UNIQUE

PRIMARY KEY

FOREIGN KEY

CHECK

DEFAULT

NOT NULL Constraint

Siguron që një kolonë **nuk mund të ketë vlera bosh** (NULL).

Garanton që të dhënat e rëndësishme janë gjithmonë të pranishme.

Shembull:

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name NVARCHAR(50) NOT NULL,     -- Emri është i detyrueshëm
    Email NVARCHAR(100) NULL        -- Email mund të jetë bosh
);
```

UNIQUE Constraint

Siguron që vlerat në një kolonë **janë unike dhe nuk përsëriten**.

Shembull:

```
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    SKU NVARCHAR(50) UNIQUE,        -- Kodi SKU duhet të jetë unik
    Name NVARCHAR(100) NOT NULL
);
```

Kolona SKU **nuk lejon dy produkte me të njëjtin kod**.

PRIMARY KEY Constraint

Kombinon UNIQUE dhe NOT NULL për të identifikuar në mënyrë unike çdo rresht.

Shembull:

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,        -- Identifikues unik për çdo porosi
    OrderDate DATE NOT NULL
);
```

Çdo rresht në tabelën Orders duhet të ketë një OrderID unik dhe jo bosh.

CHECK Constraint

Kufizon vlerat e një kolone për të përmbushur një kusht specifik.

Shembull:

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Age INT CHECK (Age >= 18),     -- Mosha duhet të jetë së paku 18
    Salary DECIMAL(10, 2) CHECK (Salary > 0) -- Paga duhet të jetë pozitive
);
```

Siguron që të dhënat në kolonat **Age** dhe **Salary** janë **të vlefshme**.

DEFAULT Constraint

Vendos një **vlerë të paracaktuar** për një kolonë **kur nuk jepet një vlerë gjatë insertimit të të dhënave**.

Shembull:

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    RegistrationDate DATE DEFAULT GETDATE()  -- Data aktuale si e paracaktuar
);
```

Nëse nuk vendoset një datë për RegistrationDate, **përdoret automatikisht data aktuale**.

Pjesëmarrja e Entiteteve

Pjesëmarrja e Entiteteve

Pjesëmarrje e Plotë (Total Participation):

- Çdo instancë në tabelën fëmijë duhet të ketë një lidhje me një instancë në tabelën prind.
- Implementohet **duke përdorur NOT NULL** me çelësin e huaj.

Pjesëmarrje e Pjesshme (Partial Participation):

- Disa instanca në tabelën fëmijë mund të mos kenë një lidhje me një instancë në tabelën prind.
- Implementohet **duke lejuar vlera NULL** për çelësin e huaj.

```sql
CREATE TABLE Enrollment (
    StudentID INT NOT NULL,                    -- Çelësi i huaj është i detyrueshëm
    CourseID INT NOT NULL,                     -- Kursi gjithashtu është i detyrueshëm
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

Pjesëmarrje e Plotë

Pjesëmarrje e Pjesshme

```sql
CREATE TABLE Enrollment (
    StudentID INT,                             -- Çelësi i huaj mund të jetë bosh
    CourseID INT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

Referencial Integrity: Cascading Actions

DELETE CASCADE

Automatikisht **fshin rreshtat e varur kur rreshti prind fshihet**.

UPDATE CASCADE

Automatikisht **përditëson rreshtat e varur kur çelësi i rreshtit prind ndryshon**.

Implementimi në SQL:

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,                        -- Identifikues unik i porosisë
    CustomerID INT,                                 -- Referenca tek klientët
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
    ON DELETE CASCADE                               -- Fshi porositë kur klienti fshihet
    ON UPDATE CASCADE                               -- Përditëso porositë kur ID e klientit
);
```

Entitetet e Dobëta

Entitetet e Dobëta (Weak Entities)

Entitete që **nuk mund të identifikohen unikisht** pa një entitet prind.

Përdorin një **çelës të përbërë që përfshin atributet e tyre dhe një çelës të huaj** për të siguruar lidhjen me entitetin prind.

Pjesëmarrja e tyre zakonisht **është e plotë**.

Implementimi në SQL:

```
CREATE TABLE Dependents (
    DependentID INT,
    EmployeeID INT,
    Name NVARCHAR(50),
    PRIMARY KEY (DependentID, EmployeeID),
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
    ON DELETE CASCADE
);
```

Kardinaliteti

Llojet e Kardinalitetit

One-to-One (Një më Një):

```
CREATE TABLE Person (
    PersonID INT PRIMARY KEY
);

CREATE TABLE Passport (
    PassportID INT PRIMARY KEY,
    PersonID INT UNIQUE,                    -- PersonID lidhet vetëm me një pasaportë
    FOREIGN KEY (PersonID) REFERENCES Person(PersonID)
);
```

One-to-Many (Një më Shumë):

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,                         -- Një klient mund të ketë shumë porosi
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Many-to-Many (Shumë më Shumë):

```
CREATE TABLE StudentCourses (
    StudentID INT,
    CourseID INT,
    PRIMARY KEY (StudentID, CourseID),      -- Çelës i përbërë për lidhjen
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);
```

Modifikimi dhe Fshirja e Tabelave

Modifikimi i Tabelave

Shtimi i Kolonave (Adding Columns):

```
ALTER TABLE Students
ADD Email NVARCHAR(100);
```

Fshirja e Kolonave (Dropping Columns):

```
ALTER TABLE Students
DROP COLUMN Email;
```

Ndryshimi i Kolonave (Modifying Columns):

```
ALTER TABLE Students
ALTER COLUMN Email NVARCHAR(150);
```

Fshirja e Tabelave - TRUNCATE vs DROP

TRUNCATE

Fshin të gjitha të dhënat në tabelë, por ruan strukturën e saj.

```
TRUNCATE TABLE Students;
```

DROP

Fshin tabelën dhe të gjitha të dhënat e saj, përfshirë strukturën dhe indeksat.

Nuk mund të rikthehet (tabela humbet përgjithmonë).

```
DROP TABLE Students;
```

Specializimi / Gjeneralizimi

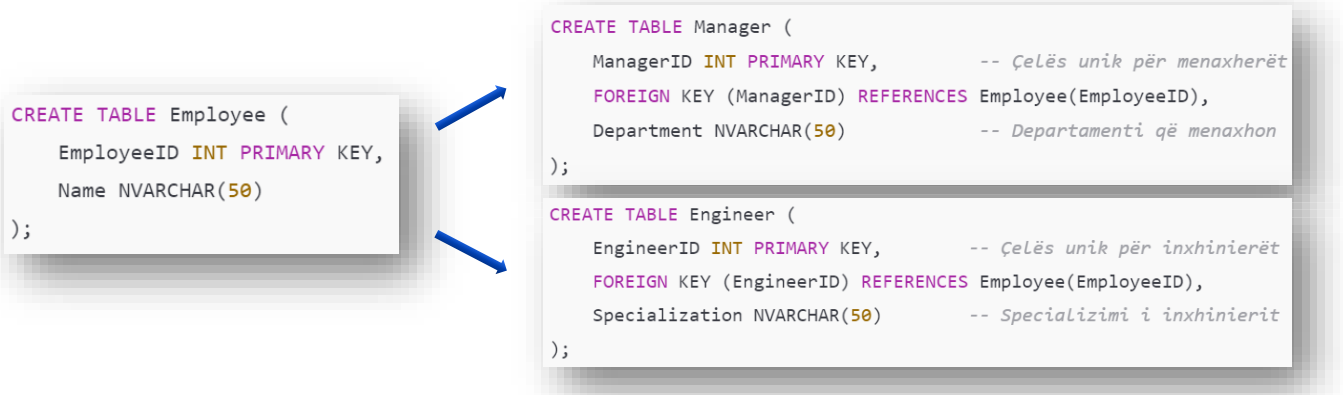
Specializimi dhe Gjeneralizimi

Specializimi

- Procesi i **përcaktimit të nëntipeve nga një entitet i përgjithshëm**.
- Përfaqëson një **ndarje hierarkike** ku entiteti prind përmban atributet e përbashkëta, dhe nëntipet përmbajnë veçori specifike.

Gjeneralizimi

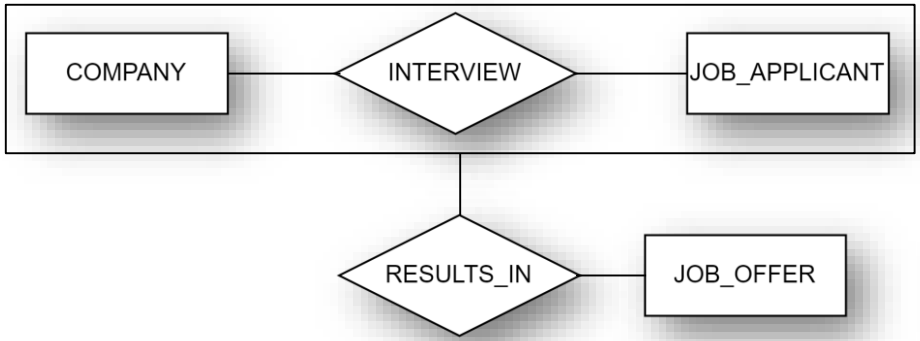
- Procesi i **kombinimit të entiteteve të shumta në një entitet të përgjithshëm**.
- Përfaqëson një **bashkim hierarkik të entiteteve** që ndajnë atribute të përbashkëta.



Agregimi

Agregimi

Agregimi përfaqëson një marrëdhënie ku një entitet lidhet me një set marrëdhëniesh në vend që të lidhet drejtpërdrejt me një entitet tjetër.




```sql
CREATE TABLE Company (
    CompanyID INT PRIMARY KEY,
    Name NVARCHAR(100)
);
```

```sql
CREATE TABLE JobOffer (
    OfferID INT PRIMARY KEY,
    Position NVARCHAR(100)
);
```

```sql
CREATE TABLE JobApplicant (
    ApplicantID INT PRIMARY KEY,
    Name NVARCHAR(100)
);
```

```sql
CREATE TABLE Interview (
    CompanyID INT,                          -- FK që lidhet me Company
    ApplicantID INT,                        -- FK që lidhet me JobApplicant
    Date DATE,                              -- Data e intervistës
    PRIMARY KEY (CompanyID, ApplicantID),   -- Çelësi i përbërë
    FOREIGN KEY (CompanyID) REFERENCES Company(CompanyID),
    FOREIGN KEY (ApplicantID) REFERENCES JobApplicant(ApplicantID)
);
```

```sql
CREATE TABLE ResultsIn (
    CompanyID INT,                          -- FK që lidhet me Company
    ApplicantID INT,                        -- FK që lidhet me JobApplicant
    OfferID INT,                            -- FK që lidhet me JobOffer
    PRIMARY KEY (CompanyID, ApplicantID, OfferID), -- Çelës i përbërë
    FOREIGN KEY (CompanyID, ApplicantID) REFERENCES Interview(CompanyID, ApplicantID),
    FOREIGN KEY (OfferID) REFERENCES JobOffer(OfferID)
);
```



Do you have any questions?

elton.boshnjaku@ubt-uni.net
www.ubt-uni.net

Referencat

Fundamentals of Database Systems
Authors: Ramez Elmasri and Shamkant B. Navathe
Edition: 7th Edition
Year: 2016

Practical SQL: A Beginner's Guide to Storytelling with Data
Author: Anthony DeBarros
Edition: 1st Edition
Year: 2018

SQL All-in-One For Dummies
Author: Allen G. Taylor
Edition: 3rd Edition
Year: 2019